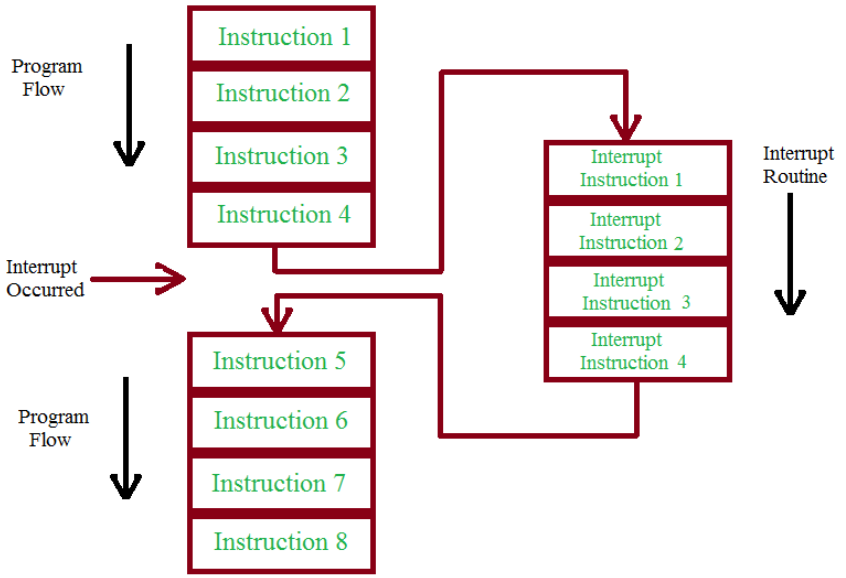
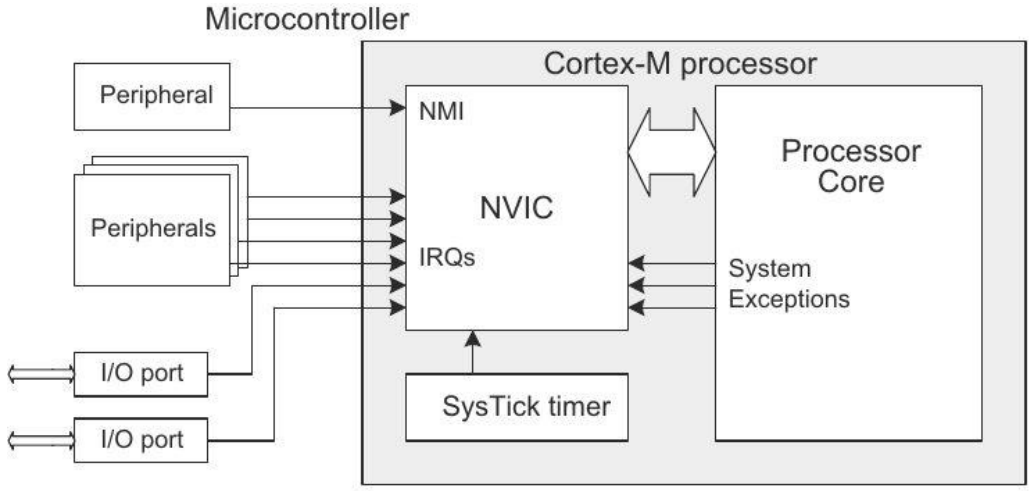
INTERRUPT

26 Oct 2022

1. Interrupt - Overview

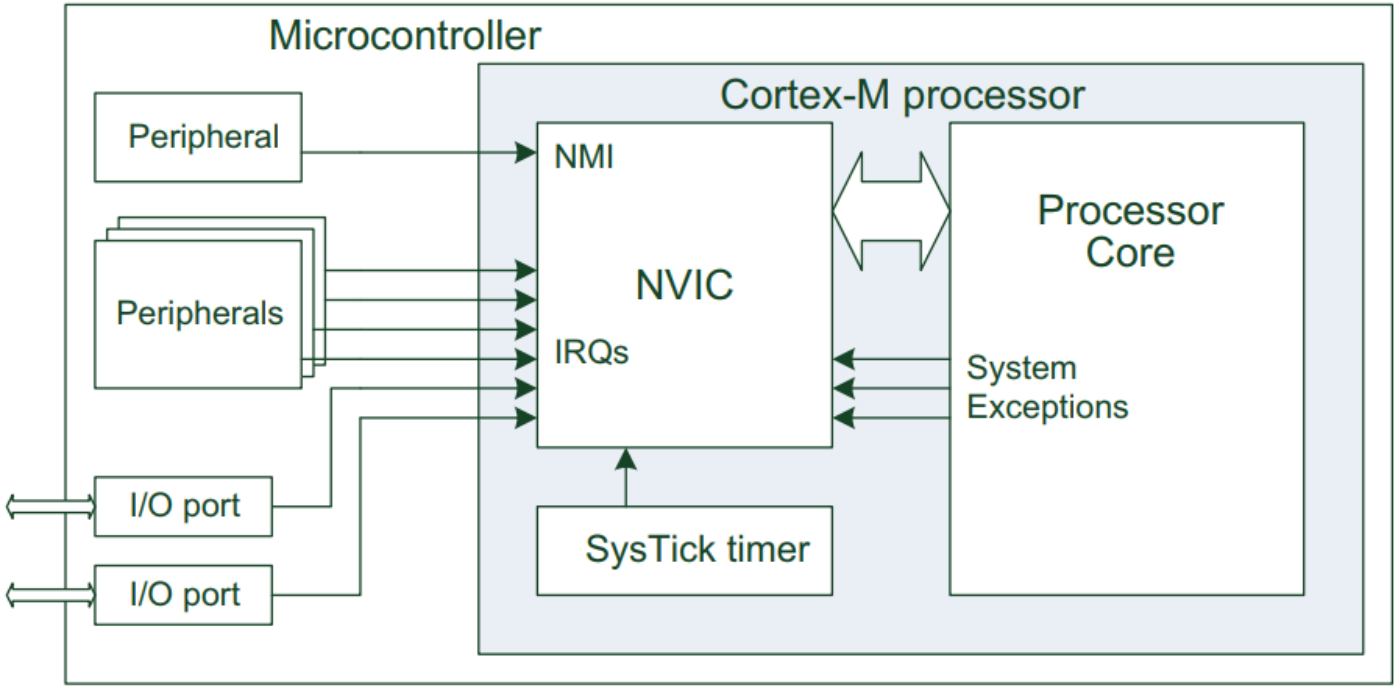
- Interrupts are a common feature available in almost all microcontrollers.
- Interrupts are events typically generated by hardware (e.g., peripherals or external input pins) that cause changes in program flow control outside a normal programmed sequence (e.g., to provide service to a peripheral).
- When a peripheral or hardware needs service from the processor, typically the following sequence would occur:
  1. The peripheral asserts an interrupt request to the processor.
  2. The processor suspends the currently executing task.
  3. The processor executes an Interrupt Service Routine (ISR) to service the peripheral, and optionally clear the interrupt request by software if needed.
  4. The processor resumes the previously suspended task.





1. Interrupt - Overview

- All Cortex-M processors provide a Nested Vectored Interrupt Controller (NVIC) for interrupt handling. In addition to interrupt requests, there are other events that need servicing and we called them “exceptions.”
- In ARM terminology, an interrupt is one type of exception.
- Other exceptions in Cortex-M processors included fault exceptions and other system exceptions to support the OS (e.g., SVC instruction).
- The pieces of program code that handle exceptions are often called exception handlers.
  In a typical Cortex-M microcontroller, the NVIC receives interrupt requests from various sources, as shown the below figure:



1. Interrupt - Overview

- The Cortex-M3 and Cortex-M4 NVIC supports up to 240 IRQs (Interrupt Requests), a Non-Maskable Interrupt (NMI), a SysTick (System Tick) timer interrupt, and a number of system exceptions.
- Most of the IRQs are generated by peripherals such as timers, I/O ports, and communication interfaces (e.g., UART, I2C).
- The NMI is usually generated from peripherals like a watchdog timer or Brown-Out Detector (BOD). The rest of the exceptions are from the processor core. Interrupts can also be generated using software

2. Exception types

- Exceptions are numbered 1-15 for system exceptions and 16 and above for interrupt inputs (inputs to the processor)
- Different Cortex-M3 or Cortex-M4 microcontrollers can have different numbers of interrupt sources (from 1-240) and different numbers of priority levels.
- This is because chip designers can configure the Cortex-M3 or Cortex-M4 design source code for different application requirements.
- The value of the currently running exception is indicated by the special register Interrupt Program Status Register (IPSR) or by one of the registers in the NVIC called the Interrupt Control State Register (the VECTACTIVE field).

Table 2-5 IPSR bit assignments

Bits	Name	Function
[31:9]	-	Reserved
[8:0]	ISR_NUMBER	This is the number of the current exception: 0 = Thread mode 1 = Reserved 2 = NMI 3 = HardFault 4 = MemManage 5 = BusFault 6 = UsageFault 7-10 = Reserved 11 = SVCall 12 = Reserved for Debug 13 = Reserved 14 = PendSV 15 = SysTick 16 = IRQ0. . . . n+15 = IRQ(n-1)[a]

2. Exception types

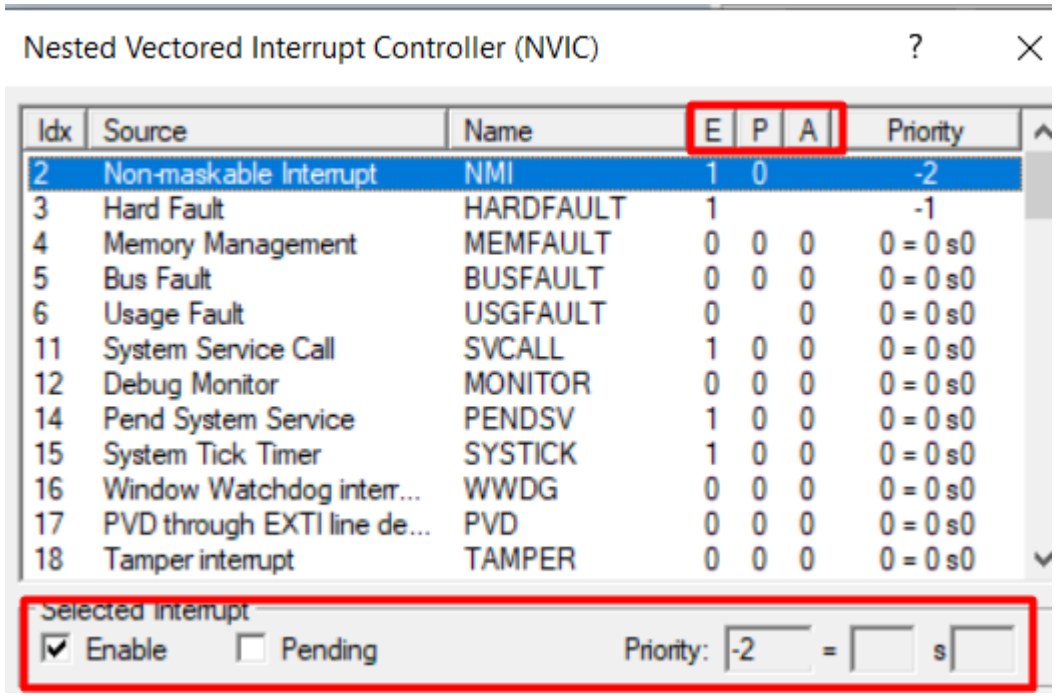
- The exception types are:
- Reset: Reset is invoked on power up or a warm reset. The exception model treats reset as a special form of exception. When reset is asserted, the operation of the processor stops, potentially at any point in an instruction. When reset is deasserted, execution restarts from the address provided by the reset entry in the vector table. Execution restarts as privileged execution in Thread mode.
- NMI: A Non Maskable Interrupt (NMI) can be signalled by a peripheral or triggered by software. This is the highest priority exception other than reset. It is permanently enabled and has of -2. NMIs cannot be: masked or prevented from activation by any other exception preemptea fixed priority d by any exception other than Reset.
- HardFault: A HardFault is an exception that occurs because of an error during exception processing, or because an exception cannot be managed by any other exception mechanism. HardFaults have a fixed priority of -1, meaning they have higher priority than any exception with configurable priority.
- SysTick: A SysTick exception is an exception the system timer generates when it reaches zero. Software can also generate a SysTick exception. In an OS environment, the processor can use this exception as system tick. Interrupt (IRQ) A interrupt, or IRQ, is an exception signalled by a peripheral, or generated by a software request. All interrupts are asynchronous to instruction execution. In the system, peripherals use interrupts to communicate with the processor.

Refer chapter 2.3.2 Exception types in Cortex-M4 Devices Generic User Guide

3. Interrupt - State

❖ Each exception is in one of the following states:

- Inactive: The exception is not active and not pending.
- Pending: The exception is waiting to be serviced by the processor. An interrupt request from a peripheral or from software can change the state of the corresponding interrupt to pending.
- Active: An exception that is being serviced by the processor but has not completed.
- Active and pending: The exception is being serviced by the processor and there is a pending exception from the same source.
- Note: An exception handler can interrupt the execution of another exception handler. In this case both exceptions are in the active state.



4. Vector table

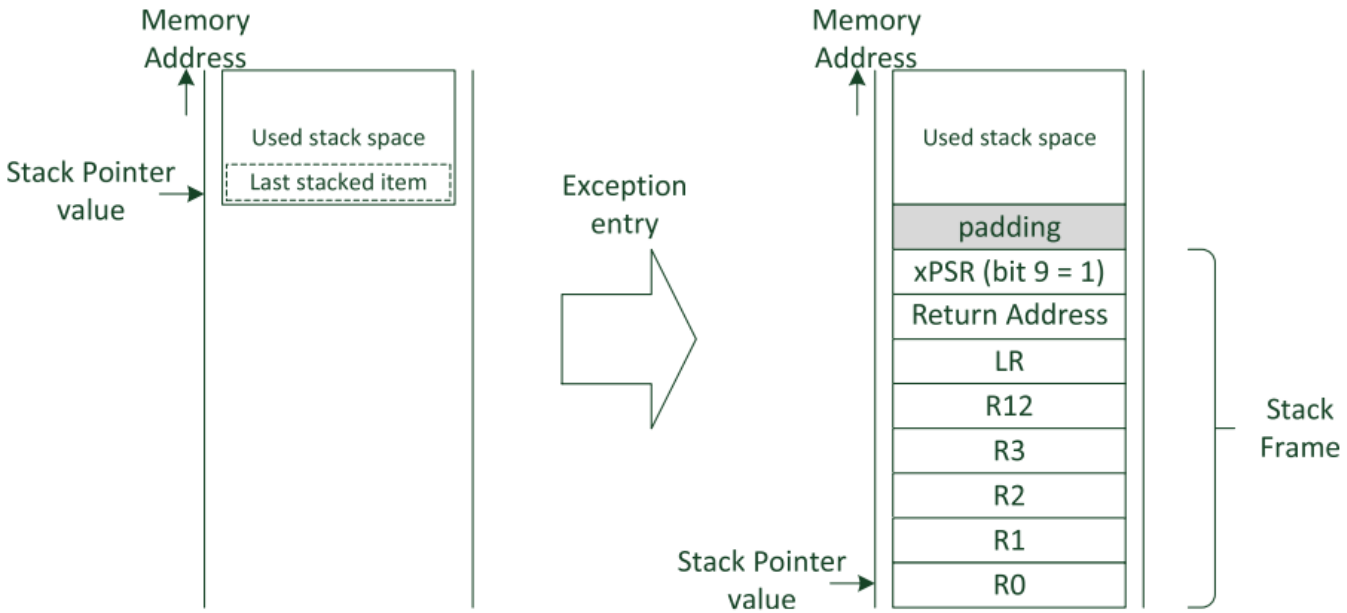
- When the Cortex-M processor accepts an exception request, the processor needs to determine the starting address of the exception handler (or ISR if the exception is an interrupt). This information is stored in the vector table in the memory.
- By default, the vector table starts at memory address 0, and the vector address is arranged according to the exception number times four. The vector table is normally defined in the startup codes provided by the microcontroller vendors.
- The Vector Table Relocation feature provides a programmable register called the Vector Table Offset Register (VTOR)

```
4
5 ; Vector Table Mapped to Address 0 at Reset
6                 AREA    RESET, DATA, READONLY
7                 EXPORT  __Vectors
8                 EXPORT  __Vectors_End
9                 EXPORT  __Vectors_Size
0
1 __Vectors       DCD     __initial_sp              ; Top of Stack
2                 DCD     Reset_Handler             ; Reset Handler
3                 DCD     NMI_Handler               ; NMI Handler
4                 DCD     HardFault_Handler         ; Hard Fault Handler
5                 DCD     MemManage_Handler         ; MPU Fault Handler
```

Exception number	IRQ number	Offset	Vector
16+n	n	0x0040+4n	IRQn
.	.	.	.
.	.	.	.
.	.	.	.
		0x004C	
18	2	0x0048	IRQ2
17	1	0x0044	IRQ1
16	0	0x0040	IRQ0
15	-1	0x003C	Systick
14	-2	0x0038	PendSV
13			Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			
9			Reserved
8			
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

5. Stacking and Unstacking

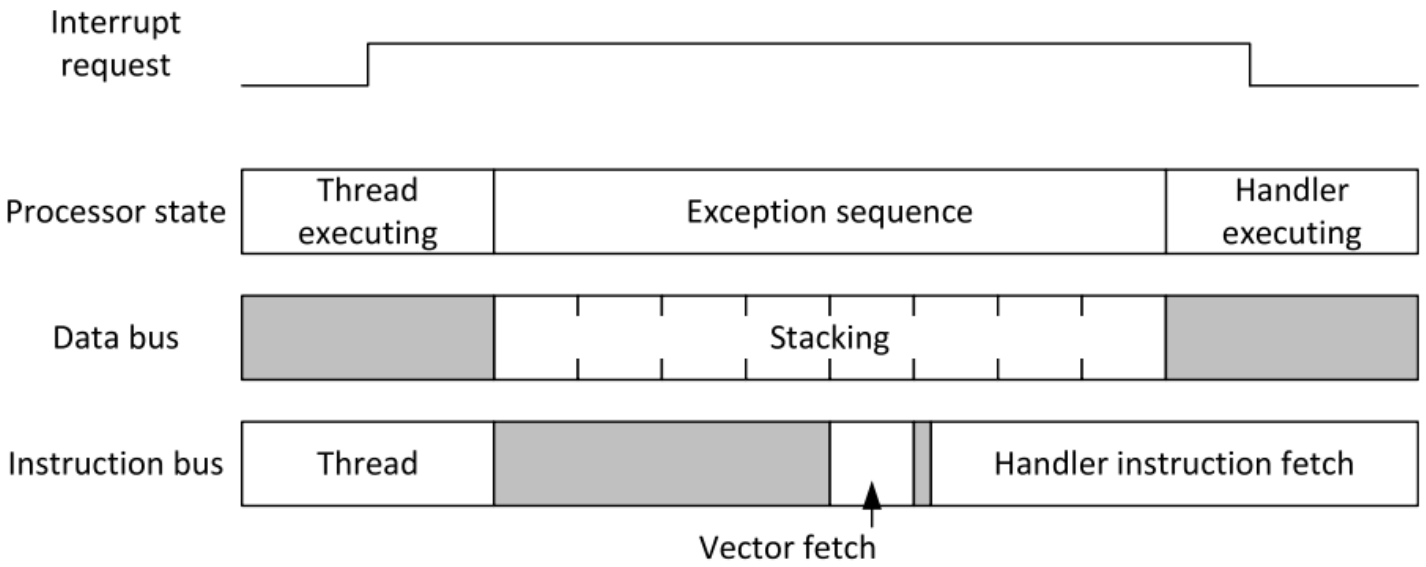
- The interrupt entry sequence, a number of registers are pushed onto the stack automatically. This is called stacking.
- When the exception return mechanism is triggered, the processor accesses the previously stacked register values in the stack memory during exception entrance and restores them back to the register bank. This is called unstacking.



6. Exception sequence - Exception entrance sequence

❖ An exception entrance sequence contains several operations:

1. Stacking of a number of registers, including return address to the currently selected stack. This enables an exception handler to be written as a normal C function.
2. Fetching the exception vector (starting address of the exception handler/ISR). This can happen in parallel to the stacking operation to reduce latency.
3. Fetching the instructions for the exception handler to be executed. After the starting address of the exception handler is determined, the instructions can be fetched.
4. Update of various NVIC registers and core registers. This includes the pending status and active status of the exception, and registers in the processor core including the Program Status Register (PSR), Link Register (LR), Program Counter (PC), and Stack Pointer (SP).



6. Exception sequence - Exception handler execution

- Within the exception handler, you can carry out services for the peripheral that requires service. The processor is in Handler mode when executing an exception handler. In Handler mode:
  • The Main Stack Pointer (MSP) is used for stack operations
  • The processor is executing in privileged access level
- If a higher-priority exception arrives during this stage, the new interrupt will be accepted, and the currently executing handler will be suspended and pre-empted by the higher-priority handler. This is called a nested exception.
- If another exception with the same or lower priority arrives during this stage, the newly arrived exception will stay in the pending state and will be serviced when the current exception handler is completed.

6. Exception sequence - Exception return

- In ARM Cortex-M processors, the exception return mechanism is triggered using a special return address called EXC_RETURN. This value is generated at exception entrance and is stored in the Link Register (LR). When this value is written to the PC with one of the allowed exception return instructions, it triggers the exception return sequence.
- When the exception return mechanism is triggered, the processor accesses the previously stacked register values in the stack memory during exception entrance and restores them back to the register bank.
- In parallel to the unstacking operation, the processor can start fetching the instructions of the previously interrupted program to allow the program to resume operation as soon as possible.

6. Exception sequence - Exception return

Table 8.1 Bit Fields of the EXC_RETURN

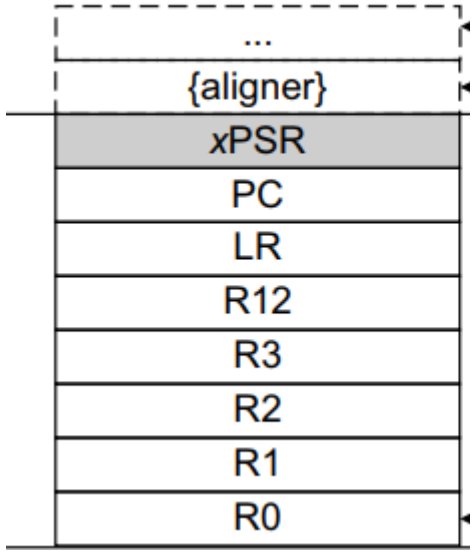
Bits	Descriptions	Values
31:28	EXC_RETURN indicator	0xF
27:5	Reserved (all 1)	0xEFFFFF (23 bits of 1)
4	Stack Frame type	1 (8 words) or 0 (26 words). Always 1 when the floating unit is unavailable. This value is set to the inverted value of FPCA bit in the CONTROL register when entering an exception handler.
3	Return mode	1 (Return to Thread) or 0 (Return to Handler)
2	Return stack	1 (Return with Process Stack) or 0 (Return with Main Stack)
1	Reserved	0
0	Reserved	1

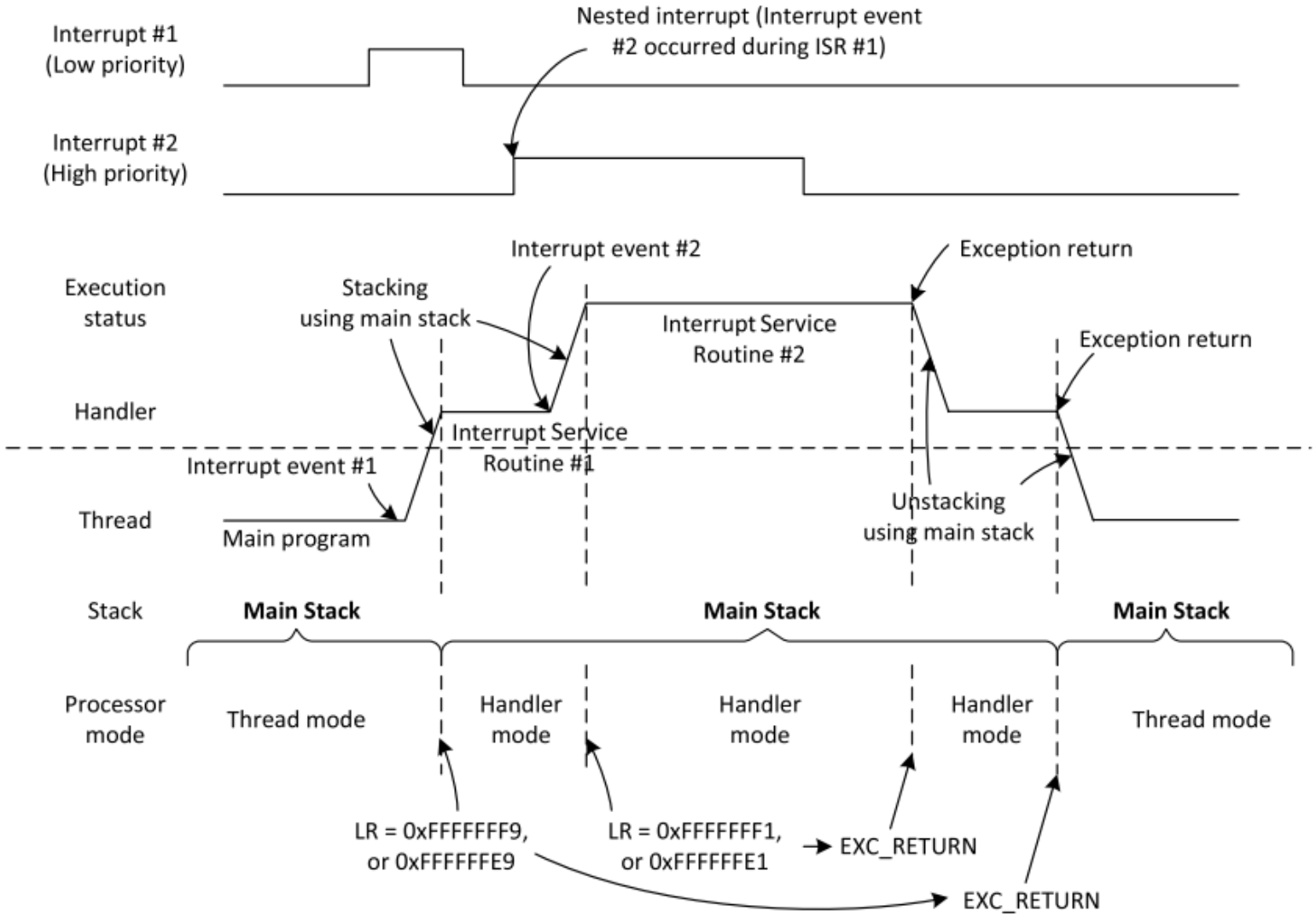
6. Exception sequence - Exception return

Table 8.2 Valid Values for EXC_RETURN

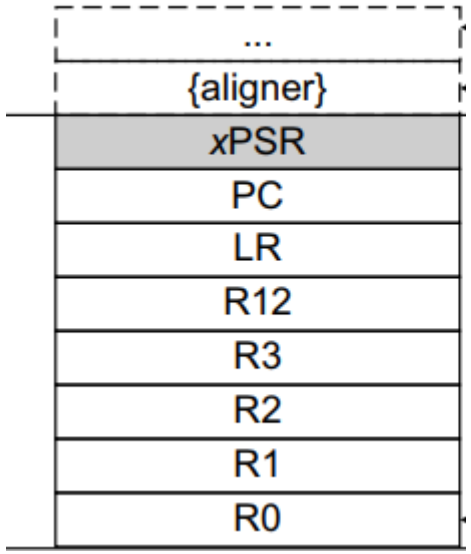
	Floating Point Unit was used before Interrupt (FPCA = 1)	**Floating Point Unit was not used before Interrupt (FPCA = 0)**
Return to Handler mode (always use Main Stack)	0xFFFFFFE1	0xFFFFFFF1
Return to Thread mode and use the Main Stack for return	0xFFFFFFE9	0xFFFFFFF9
Return to Thread mode and use the Process Stack for return	0xFFFFFFED	0xFFFFFFFD

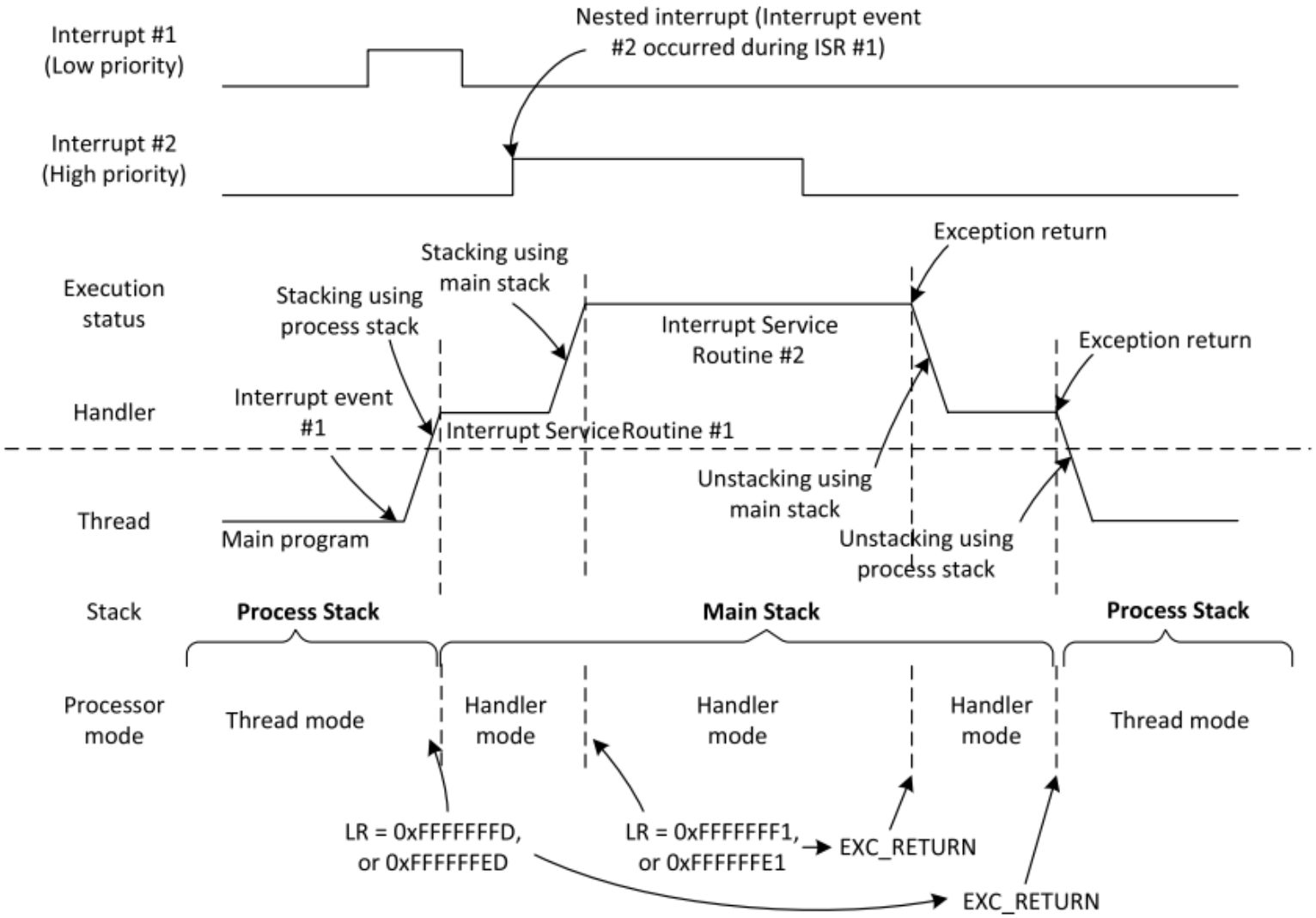
7. Example


...
{aligner}
xPSR
PC
LR
R12
R3
R2
R1
R0


Interrupt #1
(Low priority)
Nested interrupt (Interrupt event
#2 occurred during ISR #1)
Interrupt #2
(High priority)
Execution
status
Interrupt event #2
Stacking
using main stack
Interrupt Service
Routine #2
Exception return
Exception return
Handler
Interrupt Service
Routine #1
Interrupt event #1
Thread
Main program
Unstacking
using main stack
Stack
Main Stack
Main Stack
Main Stack
Processor
mode
Thread mode
Handler
mode
Handler
mode
Handler
mode
Thread mode
LR = 0xFFFFFFF9,
or 0xFFFFFFE9
LR = 0xFFFFFFF1,
or 0xFFFFFFE1
EXC_RETURN
EXC_RETURN

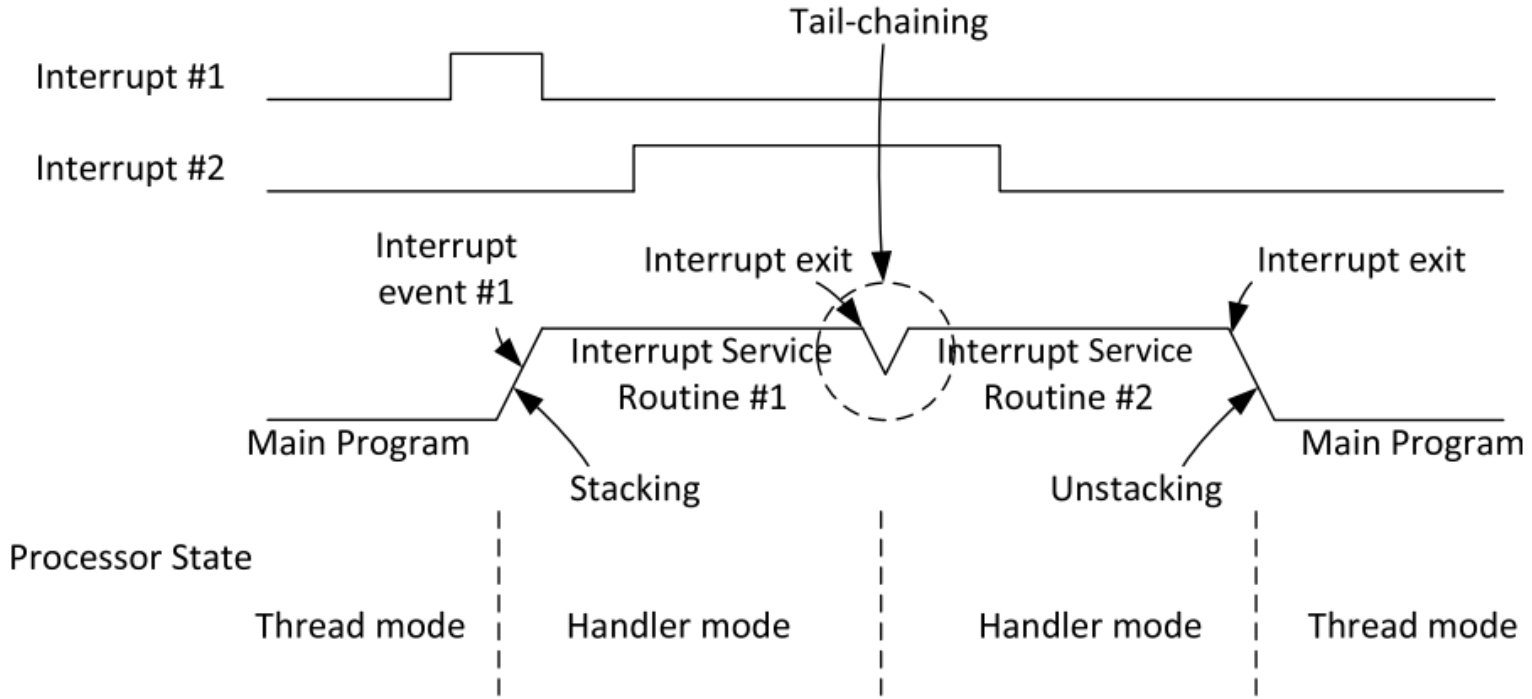
7. Example


...
{aligner}
xPSR
PC
LR
R12
R3
R2
R1
R0


Interrupt #1
(Low priority)
Nested interrupt (Interrupt event
#2 occurred during ISR #1)
Interrupt #2
(High priority)
Execution
status
Handler
Thread
Stack
Processor
mode
Stacking using
process stack
Stacking using
main stack
Interrupt event
#1
Interrupt Service Routine #1
Interrupt Service
Routine #2
Exception return
Exception return
Unstacking using
main stack
Unstacking using
process stack
Main program
Process Stack
Main Stack
Process Stack
Thread mode
Handler
mode
Handler
mode
Handler
mode
Thread mode
LR = 0xFFFFFFFD,
or 0xFFFFFFED
LR = 0xFFFFFFF1,
or 0xFFFFFFE1
EXC_RETURN
EXC_RETURN

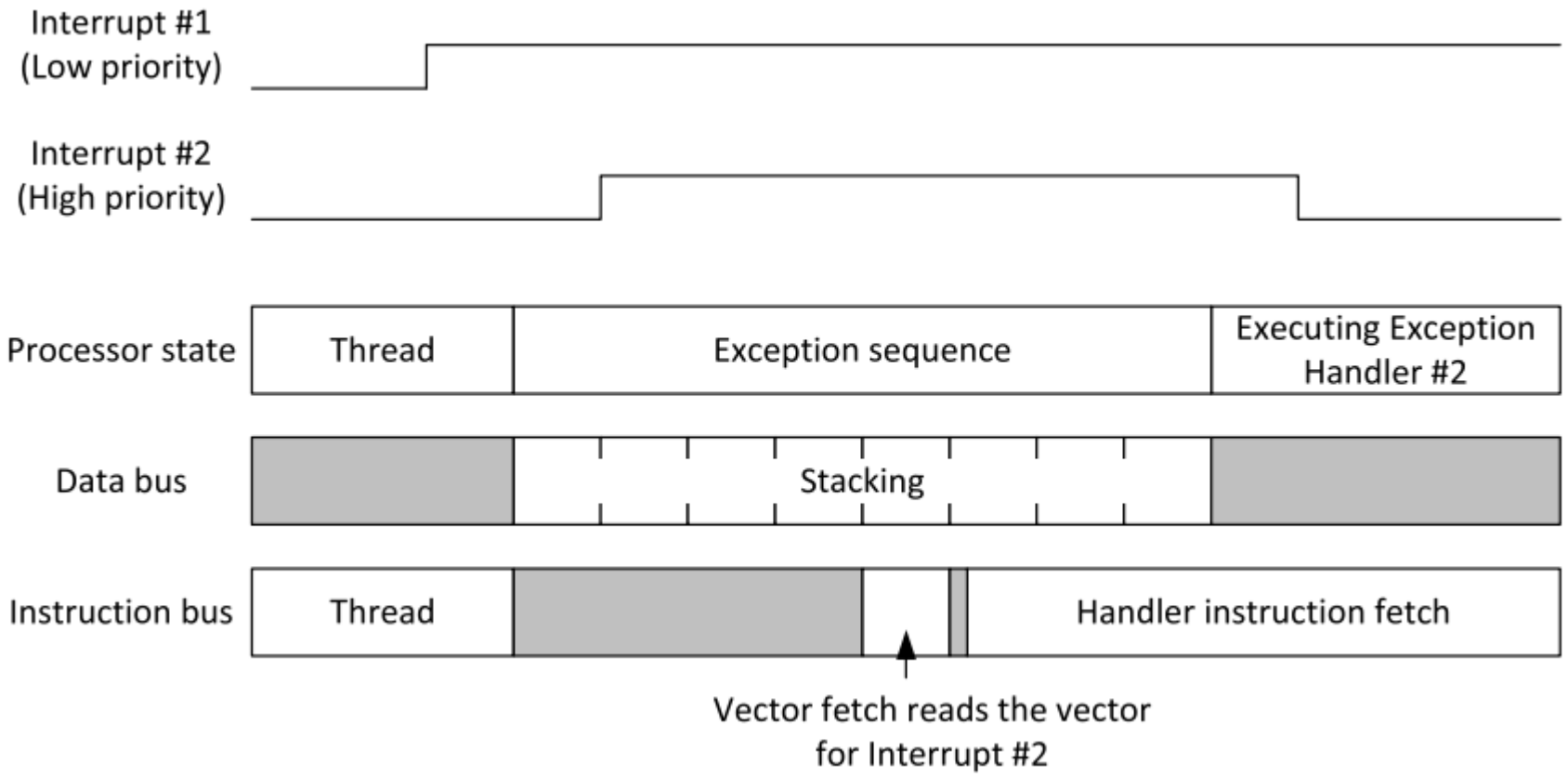
8. Tail chaining

- When an exception takes place but the processor is handling another exception of the same or higher priority, the exception will enter the pending state. When the processor finishes executing the current exception handler, it can then proceed to process the pending exception/interrupt request. Instead of restoring the registers back from the stack (unstacking) and then pushing them on to the stack again (stacking), the processor skips the unstacking and stacking steps and enters the exception handler of the pended exception as soon as possible In this way, the timing gap between the two exception handlers is considerably reduced.



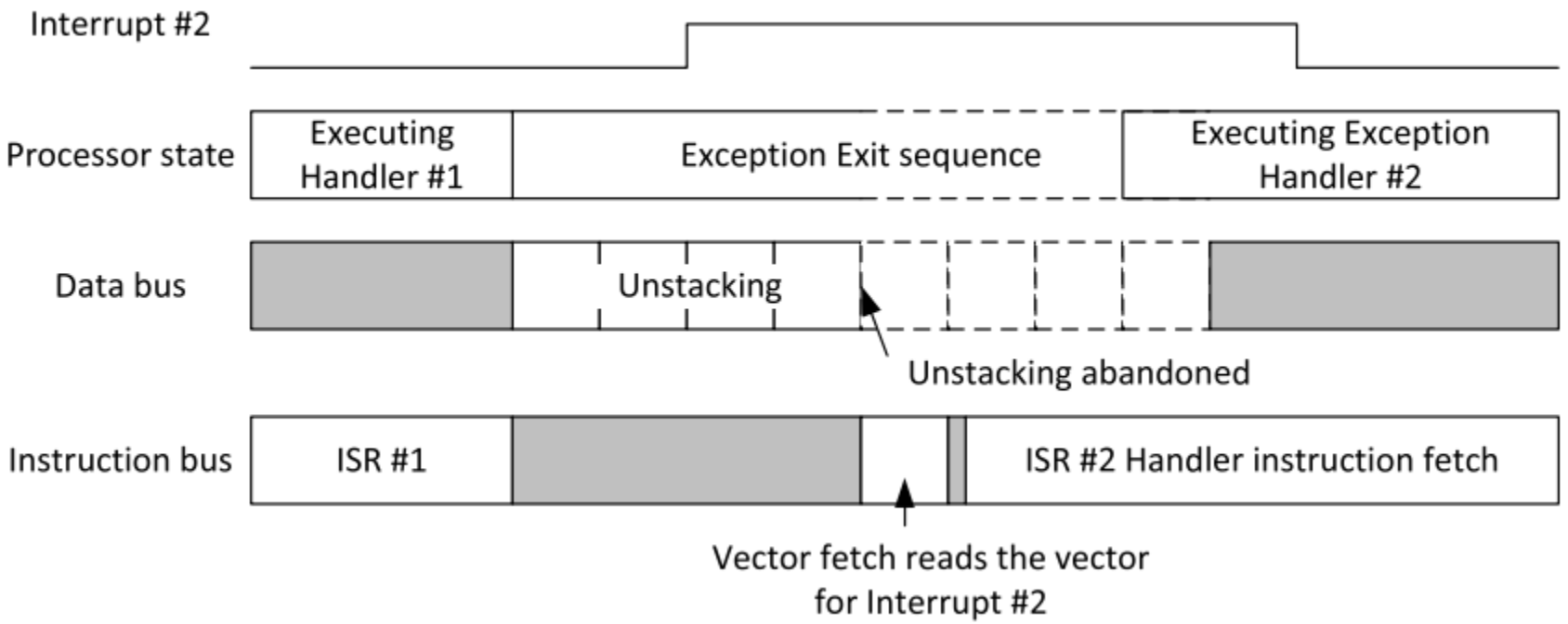
9. Late arrival

- When an exception takes place, the processor accepts the exception request and starts the stacking operation. If during this stacking operation another exception of higher priority takes place, the higher priority late arrival exception will be serviced first.



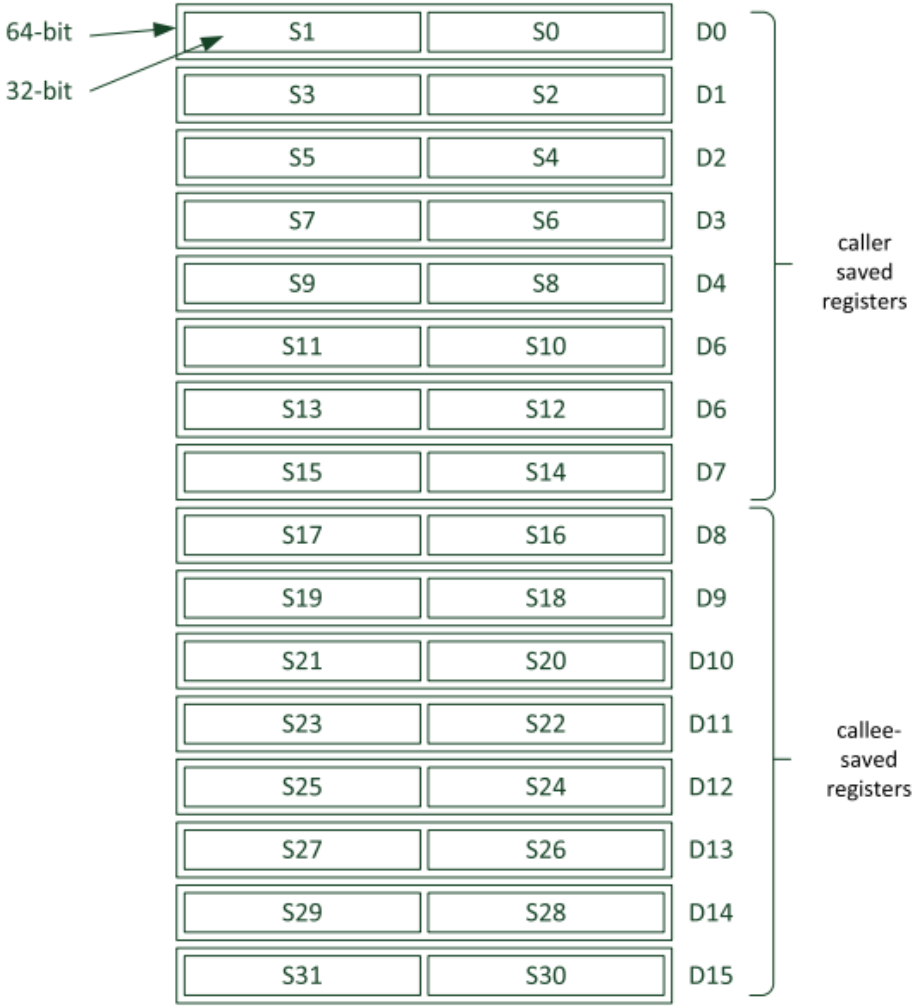
10. Pop preemption

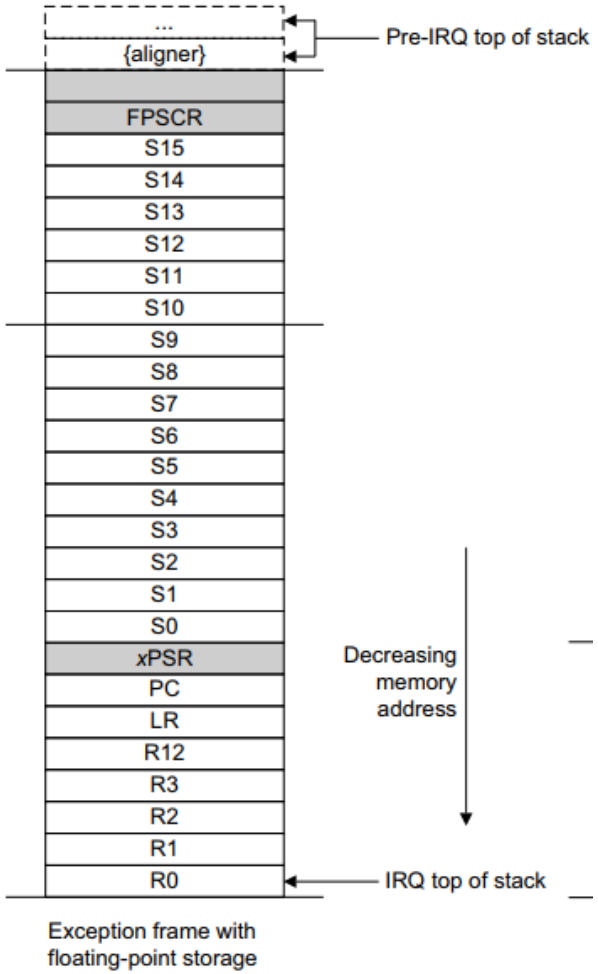
- If an exception request arrives during the unstacking process of another exception handler that has just finished, the unstacking operation would be abandoned and the vector fetch and instruction fetch for the next exception service begins. This optimization is called pop pre-emption.



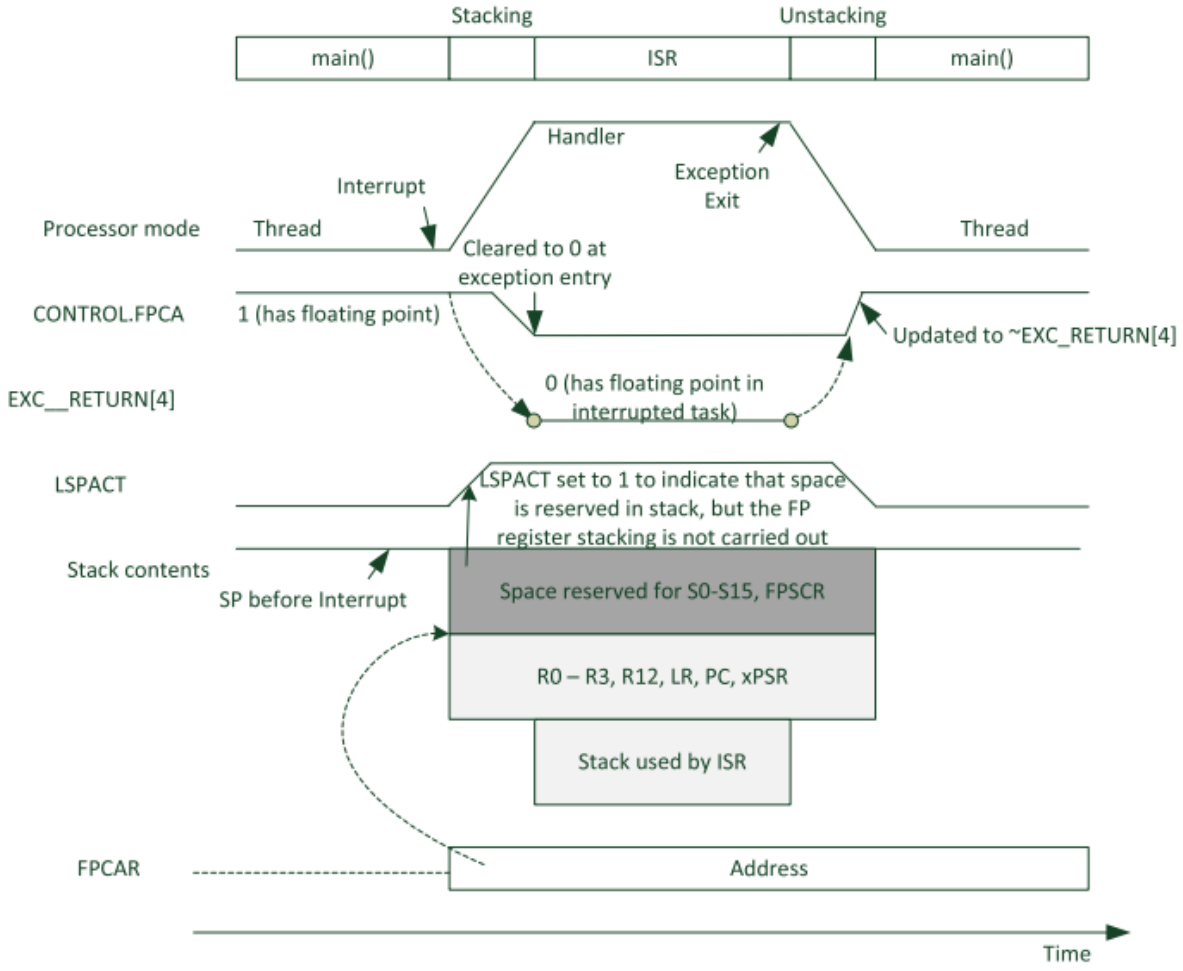
11. Lazy stacking

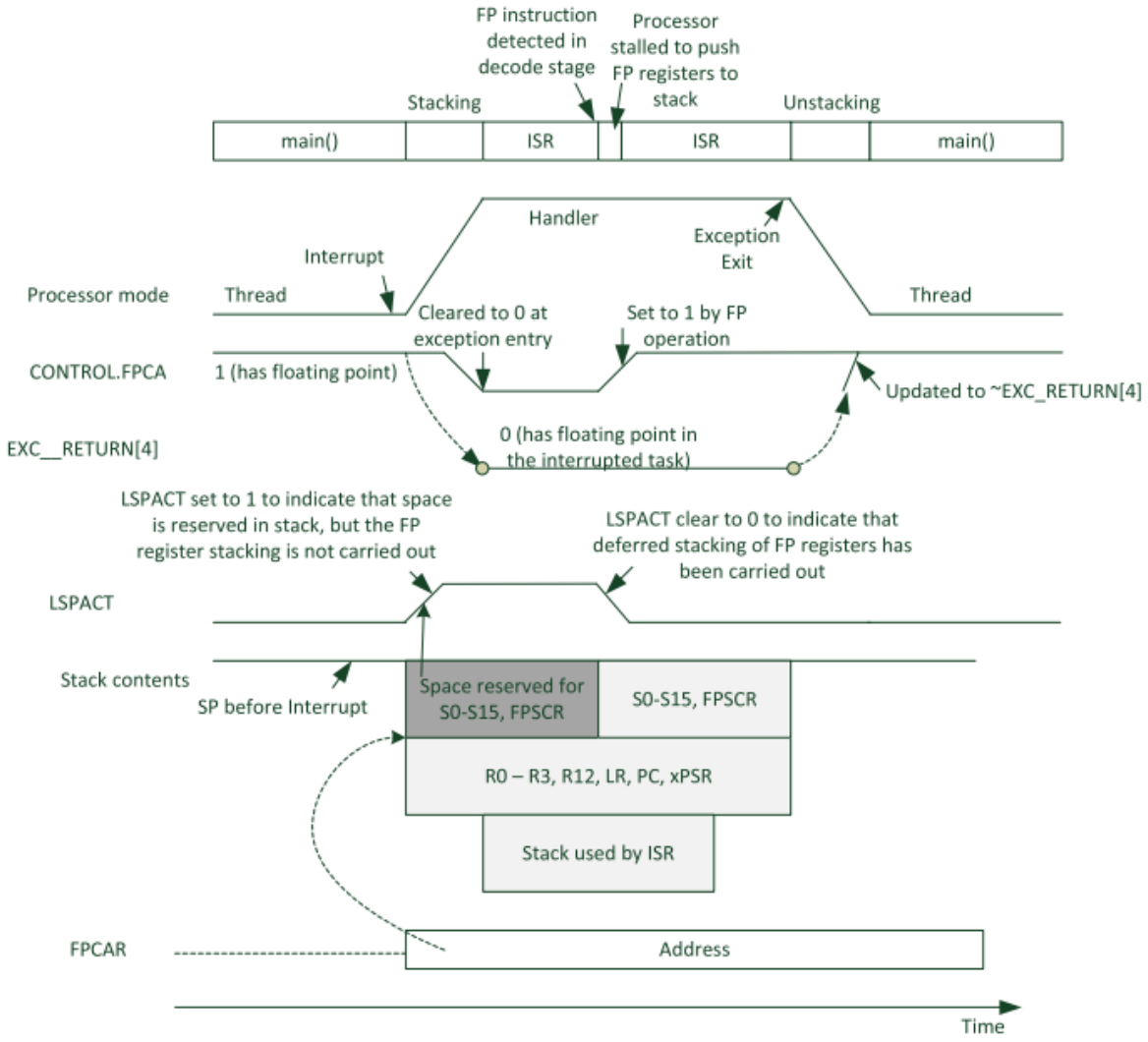
❖ Floating point register bank





11. Lazy stacking





12. Nested Vectored Interrupt Controller

There are a number of registers in the NVIC for interrupt control (exception type 16 up to 255). These registers are located in the System Control Space (SCS) address range.
All of these registers, with the exception of the Software Trigger Interrupt Register (STIR), can only be accessed in privileged access level
After reset, all interrupts are disabled and given a priority-level value of 0. Before using any interrupts, you need to:

- Set up the priority level of the required interrupt (this step is optional)
- Enable the interrupt generation control in the peripheral that triggers the interrupt

Clear pending status

- Enable the interrupt in the NVIC

you might
need to clear the interrupt request from the peripheral within the handler

12. Nested Vectored Interrupt Controller

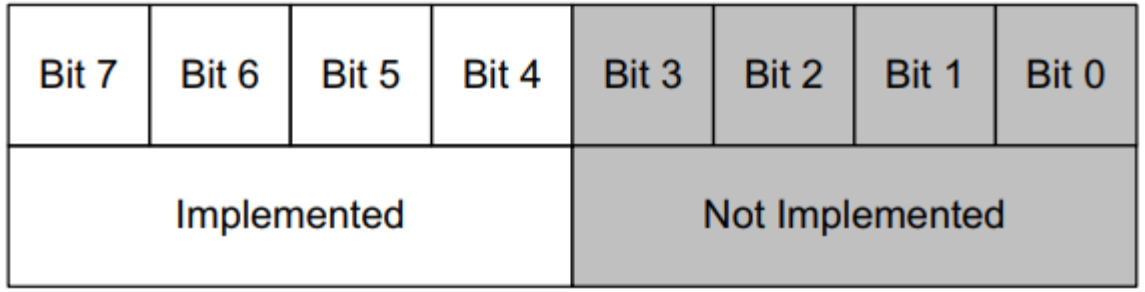
Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	*Interrupt Set-enable Registers* on page 4-4
0XE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	*Interrupt Clear-enable Registers* on page 4-5
0XE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	*Interrupt Set-pending Registers* on page 4-5
0XE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	*Interrupt Clear-pending Registers* on page 4-6
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	*Interrupt Active Bit Registers* on page 4-7
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	*Interrupt Priority Registers* on page 4-7
0xE000EF00	STIR	WO	Configurable[a]	0x00000000	*Software Trigger Interrupt Register* on page 4-8

12. Priority and Group priority

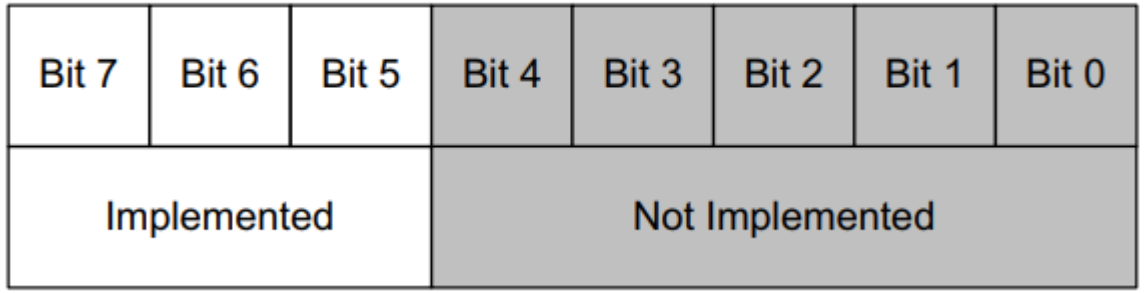
❖ Interrupts priority:

- When an exception can be accepted by the processor and get its handler executed can be dependent on the priority of the exception and the current priority of the processor.
- A higher-priority (smaller number in priority level) exception can pre-empt a lower-priority (larger number in priority level) exception
- Other exceptions have programmable priority levels, which range from 0 to 255
- The design of the Cortex-M3 and Cortex-M4 processors support three fixed highest-priority levels and up to 256 levels of programmable priority (with a maximum of 128 levels of preemption).
- The actual number of available programmable priority levels is decided by silicon chip designers. Most Cortex-M3 or Cortex-M4 chips have fewer supported levels e for example, 8, 16, 32, and so on.

- A priority-level register with 4 bits implemented (16 programmable priority levels)



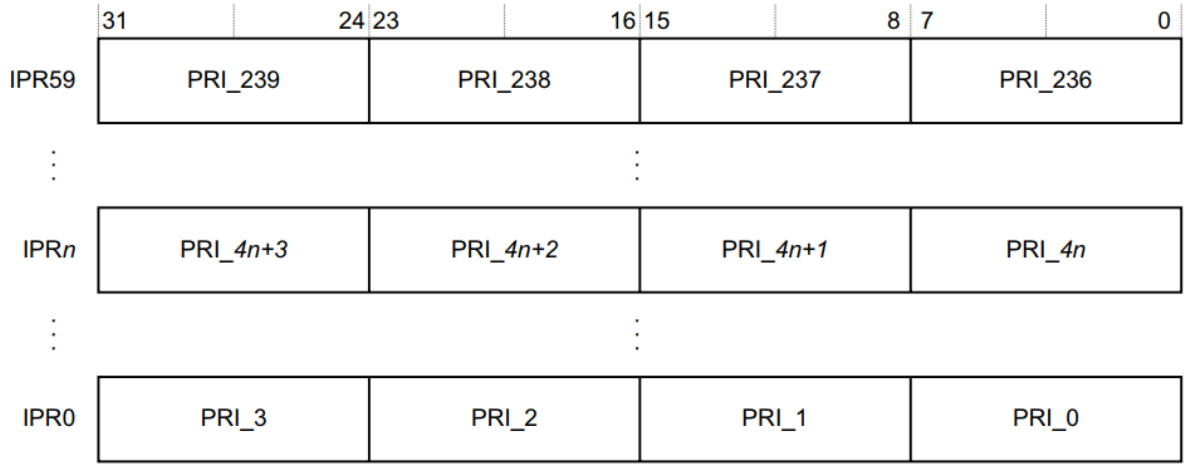
- A priority-level register with 3 bits implemented (8 programmable priority levels)



12. Priority and Group priority

❖ Interrupts priority register:

Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	*Interrupt Set-enable Registers* on page 4-4
0XE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	*Interrupt Clear-enable Registers* on page 4-5
0XE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	*Interrupt Set-pending Registers* on page 4-5
0XE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	*Interrupt Clear-pending Registers* on page 4-6
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	*Interrupt Active Bit Registers* on page 4-7
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	*Interrupt Priority Registers* on page 4-7
0xE000EF00	STIR	WO	Configurable[a]	0x00000000	*Software Trigger Interrupt Register* on page 4-8



❖ Configure a Interrupts with the NVIC interrups ID is 61 with the priority is 255

- Enable interrups: *(unsigned int *)(0xE000E100UL + ((61UL >> 5U) << 2U)) |= (1UL << (61UL & 31UL));
- Configure priority(optional): *(unsigned int *)(0xE000E400UL + ((61UL >> 2U) << 2U)) |= (255U << ((61UL & 3UL)<<3U));
- Software trigger interrupts:

*(unsigned int *)(0xE000E200UL + ((61UL >> 5U) << 2U)) |= (1UL << (61UL & 31UL));

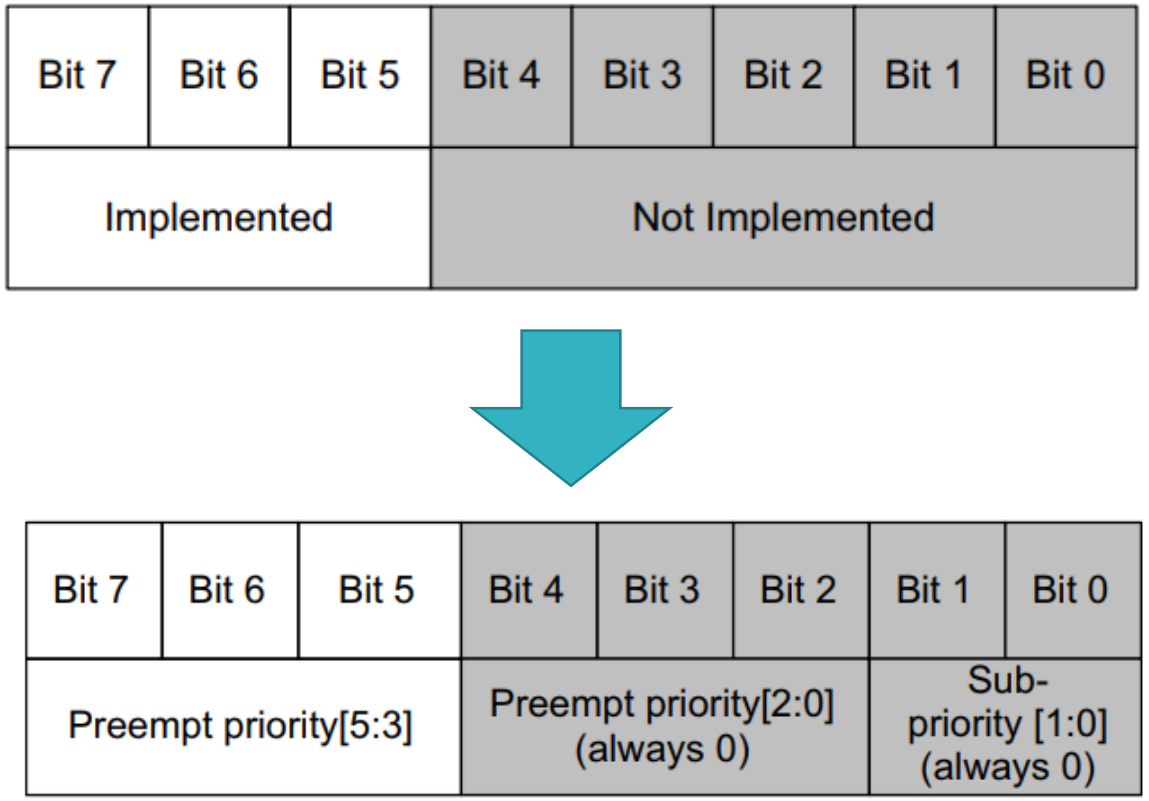
Or:

*(unsigned int *)(0xE000EF00UL) = 61UL;

12. Priority and Group priority

❖ Group priority:

- The 8-bit register which configure the interrupt priority is further divided into two parts: group priority(Pre-emption Priority) and sub-priority. The upper half (left bits) is the group (pre-empt) priority, and the lower half (right bits) is the sub-priority.
- The group priority level defines whether an interrupt can take place when the processor is already running another interrupt handler.
- The sub-priority level value is used only when two exceptions with same group-priority level occur at the same time. In this case, the exception with higher sub-priority (lower value) will be handled first.
- When two interrupts are asserted at the same time with exactly the same group/pre-empt priority level and sub-priority level, the interrupt with the smaller exception number has higher priority. (IRQ #0 has higher priority than IRQ #1.)
- The configure the group priority is configured in *System Control Block* (SCB)



13. System Control Block (SCB)

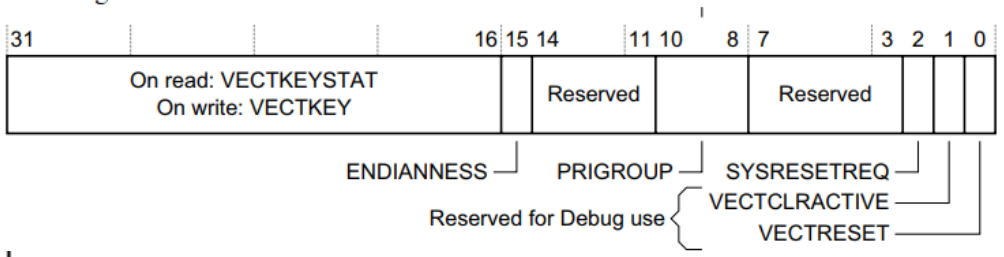
❖ The *System Control Block* (SCB) provides system implementation information, and system control. This includes configuration, control, and reporting of the system exceptions. The system control block registers are:

Address	Name	Type	Required privilege	Reset value	Description
0xE000E008	ACTLR	RW	Privileged	0x00000000	*Auxiliary Control Register*
0xE000ED00	CPUID	RO	Privileged	0x410FC240	*CPUID Base Register* on page 4-13
0xE000ED04	ICSR	RW[a]	Privileged	0x00000000	*Interrupt Control and State Register* on page 4-13
0xE000ED08	VTOR	RW	Privileged	0x00000000	*Vector Table Offset Register* on page 4-16
0xE000ED0C	AIRCR	RW[a]	Privileged	0xFA050000	*Application Interrupt and Reset Control Register* on page 4-16
0xE000ED10	SCR	RW	Privileged	0x00000000	*System Control Register* on page 4-19
0xE000ED14	CCR	RW	Privileged	0x00000200	*Configuration and Control Register* on page 4-19
0xE000ED18	SHPR1	RW	Privileged	0x00000000	*System Handler Priority Register 1* on page 4-21
0xE000ED1C	SHPR2	RW	Privileged	0x00000000	*System Handler Priority Register 2* on page 4-22
0xE000ED20	SHPR3	RW	Privileged	0x00000000	*System Handler Priority Register 3* on page 4-22
0xE000ED24	SHCRS	RW	Privileged	0x00000000	*System Handler Control and State Register* on page 4-23
0xE000ED28	CFSR	RW	Privileged	0x00000000	*Configurable Fault Status Register* on page 4-24
0xE000ED28	MMSR[b]	RW	Privileged	0x00	*MemManage Fault Status Register* on page 4-25
0xE000ED29	BFSR[b]	RW	Privileged	0x00	*BusFault Status Register* on page 4-26
0xE000ED2A	UFSR[b]	RW	Privileged	0x0000	*UsageFault Status Register* on page 4-28
0xE000ED2C	HFSR	RW	Privileged	0x00000000	*HardFault Status Register* on page 4-30
0xE000ED34	MMAR	RW	Privileged	Unknown	*MemManage Fault Address Register* on page 4-30
0xE000ED38	BFAR	RW	Privileged	Unknown	*BusFault Address Register* on page 4-31
0xE000ED3C	AFSR	RW	Privileged	0x00000000	*Auxiliary Fault Status Register* on page 4-31

13. System Control Block (SCB)

Application Interrupt and Reset Control Register

The bit assignments are:



Bits	Name	Type	Reset Value	Description
31:16	VECTKEY	R/W	–	Access key; 0x05FA must be written to this field to write to this register, otherwise the write will be ignored; the read-back value of the upper half-word is 0xFA05
15	ENDIANNESS	R	–	Indicates endianness for data: 1 for big endian (BE8) and 0 for little endian; this can only change after a reset
10:8	PRIGROUP	R/W	0	Priority group
2	SYSRESETREQ	W	–	Requests chip control logic to generate a reset
1	VECTCLRACTIVE	W	–	Clears all active state information for exceptions; typically used in debug or OS to allow system to recover from system error (reset is safer).
0	VECTRESET	W	–	Resets the Cortex®-M3/M4 processor (except debug logic), but this will not reset circuits outside the processor. This is intended for debug operations. Do not use this at the same time as SYSRESETREQ.

Priority Group	Pre-empt Priority Field	Sub-priority Field
0 (default)	Bit [7:1]	Bit [0]
1	Bit [7:2]	Bit [1:0]
2	Bit [7:3]	Bit [2:0]
3	Bit [7:4]	Bit [3:0]
4	Bit [7:5]	Bit [4:0]
5	Bit [7:6]	Bit [5:0]
6	Bit [7]	Bit [6:0]
7	None	Bit [7:0]

To configure the group priority with value is 4:

```
volatile unsigned int temp;
temp      = *(unsigned int *)( 0xE000ED0CUL );
temp  &= ~( unsigned int )( 0xFFFFUL << 16U );
temp  |= ( (0x05FAUL << 16U ) | ( 0x04UL << 8U ) );
```

14. Special registers for exception or interruptmasking

❖ PRIMASK

- The PRIMASK register is used to disable all exceptions except NMI and HardFault. It effectively changes the current priority level to 0 (highest programmable level).
- In C programming, you can use the functions provided in CMSIS-Core to set and clear PRIMASK:
  void __enable_irq(); // Clear PRIMASK
  void __disable_irq(); // Set PRIMASK
- In assembly language programming, you can change the value of PRIMARK register using CPS (Change Processor State) instructions:
  CPSIE I ; Clear PRIMASK (Enable interrupts)
  CPSID I ; Set PRIMASK (Disable interrupts)

14. Special registers for exception or interruptmasking

❖ FAULTMASK

- The FAULTMASK is very similar to PRIMASK except that it changes the effective current priority level to -1, so that even the HardFault handler is blocked. Only the NMI exception handler can be executed when FAULTMASK is set.
- In C programming with CMSIS-compliant driver libraries, you can use the following CMSIS-Core functions to set and clear the FAULTMASK as follows:
  void __enable_fault_irq(void); // Clear FAULTMASK
  void __disable_fault_irq(void); // Set FAULTMASK to disable
- For assembly language users, you can change the current status of the FAULTMASK using CPS instructions as follows:
  CPSIE F ; Clear FAULTMASK
  CPSID F ; Set FAULTMASK
- FAULTMASK is cleared automatically upon exiting the exception handler except return from NMI handler. This characteristic provides an interesting usage for FAULTMASK: if in a lower-priority exception handler we want to trigger a higher-priority handler (except NMI), but want this higher-priority handler to start AFTER the lower-priority handler is completed, we can:
  - Set the FAULTMASK to disable all interrupts and exceptions.
  - Set the pending status of the higher-priority interrupt or exception
  - Exit the handler

  Because the pending higher-priority exception handler cannot start while theFAULTMASK is set, the higher-priority exception stays in the pending state until FAULTMASK is cleared, which happens when the lower-priority handler finishes. As a result, you can force the higher-priority handler to start after the lower-priority handler ends.

14. Special registers for exception or interruptmasking

- BASEPRI
  - The BASEPRI register defines the minimum priority for exception processing. When BASEPRI is set to a nonzero value, it prevents the activation of all exceptions with the same or lower priority level as the BASEPRI value
  - For example, if you want to block all exceptions with priority level equal to or lower than 0x60, you can write the value to BASEPRI:

```
__set_BASEPRI(0x60); // Disable interrupts with priority
                     // 0x60-0xFF using CMSIS-Core function
```

  - For users of assembly language, the same operation can be written as:

```
MOVS R0, #0x60
MSR BASEPRI, R0 ; Disable interrupts with priority
                ; 0x60-0xFF
```


Thank you